AI Time Log Multi-Agent Hackathon

A classroom hackathon to build a small multi-agent system for employee time logging, validation, review, and summary generation.

TekFrameworks

Confidentiality Notice: This document is proprietary to **TekFrameworks Consulting Pvt. Ltd.** and is intended solely for the Client's evaluation.

It may not be shared with third parties. For Client Use Only | TekFrameworks Consulting Pvt. Ltd.| © 2026 TekFrameworks

Scenario: Employees often miss time logs, enter incorrect hours, or use inconsistent project codes. Your task is to build a multi-agent assistant that can capture time logs, validate them, review anomalies, and summarize the week.

Hackathon Goal

- Build a multi-agent time log system, not a single agent.
- Use at least two specialist agents coordinated by one orchestrator agent.
- Use simple in-memory data structures; no database is required.
- Focus on working behavior and clear demo outputs rather than UI polish.

Required Multi-Agent Design

Agent	Responsibility	Expected Behavior
Orchestrator Agent	Receives user requests and delegates to specialist agents.	Decides whether the request is about logging, validation, review, or summary.
Time Entry Agent	Creates and retrieves employee time logs.	Uses tools such as create_time_log and get_time_logs.
Policy Validator Agent	Checks whether the entry follows timesheet rules.	Rejects negative hours, flags more than 12 hours, and checks missing project codes.
Summary Agent	Produces weekly summaries and project-wise breakdowns.	Turns raw logs into concise management-friendly summaries.

You may implement the specialist agents as Strands agents exposed to the orchestrator using the agents-as-tools pattern, or keep them as clearly separated agent objects called by the orchestrator logic.

Minimum Functional Requirements

Requirement	What your solution should do
Create time logs	Accept natural language requests and create a structured time-log entry with employee_id, date, project_code, hours, and description.
Validate time logs	Apply policy rules before saving or confirming an entry.
Retrieve logs	Show time logs for an employee for the current week or a given date range.
Summarize work	Generate a weekly summary with total hours, project-wise hours, key activities, and policy flags.
Maintain audit trail	Capture which agent or tool handled the request and whether the request was accepted, rejected, or flagged.

Suggested Tools

Implement these as Python functions and connect them to the appropriate specialist agent.

```
create_time_log(employee_id, date, project_code, hours, description)
get_time_logs(employee_id, start_date, end_date)
validate_time_log(employee_id, date, project_code, hours)
summarize_time_logs(employee_id, start_date, end_date)
add_audit_log(request, agent_name, action, status, notes)
```

Required Policy Rules

Rule	Expected Agent Behavior
Negative hours	Reject the request.
More than 12 hours in a day	Do not silently save. Flag it and ask for correction or confirmation.
Blank or vague project code	Reject and ask for a valid project code.
Weekend log	Flag as unusual. Confirmation may be required.
Another employee's logs	Do not expose or change another employee's logs unless manager role is explicitly handled.

Memory Requirement

Add simple memory for at least one employee preference. For example, the agent may remember the employee's usual project code or standard working hours.

User: My usual project is TIMELOG-AI. Remember this.

Agent: I will remember TIMELOG-AI as your usual project.

User: Log 8 hours for today.

Agent: I used your remembered project TIMELOG-AI and created the time log.

Expected Demo Flow

- Show the orchestrator receiving a user request.
- Show the orchestrator delegating to the Time Entry Agent or Policy Validator Agent.
- Show at least one accepted time-log entry.
- Show at least one rejected or flagged entry.
- Show a weekly summary produced by the Summary Agent.
- Show the audit log table at the end.

Sample Prompts Learners Can Try

These are optional examples. You may create your own prompts during the demo.

- My usual project is TIMELOG-AI. Remember this.
- Log 8 hours for EMP-1001 today. I worked on agent testing.
- Log 6 hours for EMP-1001 yesterday on PAYROLL-AI. I worked on API validation.
- Show my time logs for this week.
- Summarize my work for this week.
- Log -3 hours for EMP-1001 today on TIMELOG-AI.
- Log 16 hours for EMP-1001 today on TIMELOG-AI.

Suggested Notebook Structure

Section	Content
Imports and setup	Import Strands agent classes, tools, datetime, pandas, and any local helper libraries.
In-memory stores	Create time_log_store, employee_memory, and audit_log_store.
Tool functions	Create Python functions for create, retrieve, validate, summarize, and audit logging.
Specialist agents	Create Time Entry Agent, Policy Validator Agent, and Summary Agent.
Orchestrator agent	Create the main agent that delegates to the specialist agents.
Demo run	Run a few prompts and display final time_log_store and audit_log_store.

Output Expectations

- A working notebook that runs end-to-end.
- At least three agents: one orchestrator and two specialist agents.
- At least four tools or clearly separated functions.
- A visible memory example.
- A visible validation or guardrail example.
- A final audit log table.